Lab Assignment Number: 19.5

Name: D.Sandeep

HT No:2303A51665

Batch No:23

Task 1 – Python List Processing → C++ STL Conversion

Task:

Translate a Python program that filters even numbers from a list into

C++.

Instructions:

• Provide AI with Python list comprehension code.

• Convert it into C++ using vectors and loops/STL.

• Verify outputs for the same input list.

Expected Output:

• Equivalent working C++ program using STL.

```python
# generate a python code that filters even numbers from a list
from ast import main


def filter_even_numbers(numbers):
    """Filter even numbers from a list.

    Args:
        numbers (list): A list of integers.
    Returns:
        list: A list of even integers.
    """
    even_numbers = [num for num in numbers if num % 2 == 0]
    return even_numbers
# Example usage
if __name__ == "__main__":
    sample_numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    even_numbers = filter_even_numbers(sample_numbers)
    print("Even numbers:", even_numbers)
# convert the above code to c++ using vectors and loops/STL.
#include <iostream>
#include <vector>
#include <algorithm>
std::vector<int> filterEvenNumbers(const std::vector<int>& numbers) {
    std::vector<int> evenNumbers;
    for (int num : numbers) {
        if (num % 2 == 0) {
            evenNumbers.push_back(num);
        }
    }
    return evenNumbers;
}
int main() {
    std::vector<int> sampleNumbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    std::vector<int> evenNumbers = filterEvenNumbers(sampleNumbers);
    std::cout << "Even numbers: ";
    for (int num : evenNumbers) {
        std::cout << num << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

Task 2 – Java to Python OOP Translation

Task:

Convert a Java Student class into an equivalent Python class.

Instructions:

• Translate attributes, constructor, and methods properly.

• Ensure Python version uses __init__ and follows Python

conventions.

• Test both versions with sample objects.

Expected Output:

• Correct Python OOP class translated from Java

```java
# generate a java code with student class and respective attributes
public class Student {
    private String name;
    private int age;
    private String studentId;

    public Student(String name, int age, String studentId) {
        this.name = name;
        this.age = age;
        this.studentId = studentId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public String getStudentId() {
        return studentId;
    }

    public void setStudentId(String studentId) {
        this.studentId = studentId;
    }
}
```

```python
#convert the above java code to python code
class Student:
    def __init__(self, name: str, age: int, student_id: str):
        self.name = name
        self.age = age
        self.student_id = student_id

    def get_name(self) -> str:
        return self.name

    def set_name(self, name: str):
        self.name = name

    def get_age(self) -> int:
        return self.age

    def set_age(self, age: int):
        self.age = age

    def get_student_id(self) -> str:
        return self.student_id

    def set_student_id(self, student_id: str):
        self.student_id = student_id
# Example usage
student = Student("Alice", 20, "S12345")
print(student.get_name())  # Output: Alice
print(student.get_age())   # Output: 20
print(student.get_student_id())  # Output: S12345
student.set_name("Bob")
print(student.get_name())  # Output: Bob
student.set_age(21)
print(student.get_age())   # Output: 21
student.set_student_id("S54321")
print(student.get_student_id())  # Output: S54321
```

Task 3 – SQL Aggregation Query → Pandas GroupBy

Task:

Convert an SQL query with aggregation into Pandas.

Example SQL:

SELECT department, AVG(salary)

FROM employees

GROUP BY department;

Instructions:

• Translate into Pandas groupby() and mean().

• Validate using sample DataFrame.

Expected Output:

• Equivalent Pandas aggregation code.

```python
-- Create a SQL query to find the average salary of employees in each department from a table named 'employees'. Group the results by department
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
#convert the above SQL query to a Python code using pandas library
import pandas as pd
# Assuming you have a DataFrame named 'employees' with columns 'department' and 'salary'
average_salary_by_department = employees.groupby('department')['salary'].mean().reset_index()
print(average_salary_by_department)
```

Task 4 – Python File Handling → C++ File Streams

Task:

Convert a Python program that reads a text file and counts words

into C++.

Instructions:

• Translate open() usage into ifstream.

• Handle file-not-found errors properly.

• Compare outputs for the same file.

Expected Output:

• Working C++ word counter program.

```python
# generate a Python program that reads a text file and counts words
def count_words_in_file(file_path):
    try:
        with open(file_path, 'r') as file:
            text = file.read()
            words = text.split()
            word_count = len(words)
            print(f'The number of words in the file is: {word_count}')
    except FileNotFoundError:
        print(f'The file {file_path} was not found.')
    except Exception as e:
        print(f'An error occurred: {e}')
if __name__ == "__main__":
    file_path = input("Enter the path to the text file: ")
    count_words_in_file(file_path)
# Convert above  Python program that reads a text file and counts words to c++ program
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
using namespace std;
void countWordsInFile(const string& filePath) {
    ifstream file(filePath);
    if (!file.is_open()) {
        cout << "The file " << filePath << " was not found." << endl;
        return;
    }
    string word;
    int wordCount = 0;
    while (file >> word) {
        wordCount++;
    }
    cout << "The number of words in the file is: " << wordCount << endl;
    file.close();
}
int main() {
    string filePath;
    cout << "Enter the path to the text file: ";
    getline(cin, filePath);
    countWordsInFile(filePath);
    return 0;
}
```

Task 5 – Java Exception Handling → Python Try/Except

Task:

Translate a Java program using try-catch for division into Python.

Instructions:

• Convert exception syntax properly.

• Handle divide-by-zero gracefully.

• Verify both programs behave the same.

Expected Output:

• Equivalent Python error-handling program.

```
# generate a Java program using try-catch for division
public class Division {
    public static void main(String[] args) {
        int numerator = 10;
        int denominator = 0;

        try {
            int result = numerator / denominator;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Error: Division by zero is not allowed.");
        }
    }
}
# convert the above Java program to Python using try-except for division
def main():
    numerator = 10
    denominator = 0

    try:
        result = numerator / denominator
        print("Result:", result)
    except ZeroDivisionError:
        print("Error: Division by zero is not allowed.")
if __name__ == "__main__":
    main()
```